

SIC AI Chapter Quiz: 02

Student Name: Devesh Panwar

Candidate ID: SIC202500060

```
import numpy as np

# --- 1. Zero Vector ---
zero_vector = np.zeros(3)
print("1. Zero Vector (size 3):")
print(zero_vector)
print("-" * 30)

# --- 2. One Vector ---
one_vector = np.ones(3)
print("2. One Vector (size 3):")
print(one_vector)
print("-" * 30)

# --- 3. Square Matrix ---
square_matrix = np.array([[1, 2],
                           [3, 4]])
print("3. Square Matrix (2x2):")
print(square_matrix)
print("-" * 30)

# --- 4. Diagonal Matrix ---
diag_values = [5, 2, 9]
diagonal_matrix = np.diag(diag_values)
print("4. Diagonal Matrix:")
print(diagonal_matrix)
print("-" * 30)

# --- 5. Identity Matrix ---
identity_matrix = np.eye(3)
print("5. Identity Matrix (3x3):")
print(identity_matrix)
print("-" * 30)

# --- 6. Symmetric Matrix ---
symmetric_matrix = np.array([[1, 7, 3],
                              [7, 4, 5],
                              [3, 5, 0]])
print("6. Symmetric Matrix:")
print(symmetric_matrix)

# Verification: Check if it equals its transpose
is_symmetric = np.array_equal(symmetric_matrix, symmetric_matrix.T)
print(f"\nIs it symmetric? {is_symmetric}")
print("-" * 30)
```

```

1. Zero Vector (size 3):
[0. 0. 0.]
-----
2. One Vector (size 3):
[1. 1. 1.]
-----
3. Square Matrix (2x2):
[[1 2]
 [3 4]]
-----
4. Diagonal Matrix:
[[5 0 0]
 [0 2 0]
 [0 0 9]]
-----
5. Identity Matrix (3x3):
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
-----
6. Symmetric Matrix:
[[1 7 3]
 [7 4 5]
 [3 5 0]]

Is it symmetric? True
-----

```

```

import numpy as np

# --- Express stocks' price and quantity as vectors ---
p = np.array([1000000, 800000, 500000])
n = np.array([3, 4, 5])

print("1) Price Vector (p):", p)
print("   Quantity Vector (n):", n)
print("-" * 30)

# --- Calculate the amount required using NumPy operation ---
total_amount = np.dot(p, n)

print(f"2) Total Amount Required: {total_amount:,.0f} Won")

1) Price Vector (p): [1000000  800000  500000]
   Quantity Vector (n): [3 4 5]
-----
2) Total Amount Required: 8,700,000 Won

```

```

import numpy as np
from sklearn.datasets import load_digits

# --- Step 0: Load Data ---
# X contains 1797 images, each flattened into a vector of 64 pixels (8x8)
digits = load_digits()
X = digits.data

```

```

print(f"Data Loaded. Matrix Shape: {X.shape}")
print("-" * 50)

# --- Part 1: Similarity between 1st and 10th image ---
# Python uses 0-based indexing:
# 1st image = index 0
# 10th image = index 9

img1_vector = X[0]
img10_vector = X[9]

# Calculate dot product
similarity_pair = np.dot(img1_vector, img10_vector)

print(f"1) Similarity between Image 1 and Image 10: {similarity_pair}")
print("-" * 50)

# --- Part 2: Efficient Similarity for ALL images ---
# Instead of loops, we compute X dot X_Transpose.
# This creates a similarity matrix where row i, col j represents the similar
# between image i and image j.

similarity_matrix = np.dot(X, X.T)

print("2) Efficient Calculation (Matrix Multiplication):")
print(f"    Similarity Matrix Shape: {similarity_matrix.shape}")
print(f"    First 3 rows of Similarity Matrix:\n{similarity_matrix[:3, :3]}")

# Verification:
# The value at [0, 9] in the matrix should match our calculation from Part 1
matrix_val = similarity_matrix[0, 9]
print(f"\n    Verification Check (Matrix[0,9]): {matrix_val}")
print(f"    Matches Part 1? {matrix_val == similarity_pair}")

```

```

Data Loaded. Matrix Shape: (1797, 64)
-----
1) Similarity between Image 1 and Image 10: 2807.0
-----
2) Efficient Calculation (Matrix Multiplication):
    Similarity Matrix Shape: (1797, 1797)
    First 3 rows of Similarity Matrix:
[[3070. 1866. 2264.]
 [1866. 4209. 3432.]
 [2264. 3432. 4388.]]

    Verification Check (Matrix[0,9]): 2807.0
    Matches Part 1? True

```

```

import numpy as np

# --- Define the Matrix ---
A = np.array([[2, 0],
              [0, 1]])

print("Original Matrix A:")

```

```

print(A)
print("-" * 30)

# --- Calculate Inverse ---
# We use np.linalg.inv() to compute the multiplicative inverse of a matrix.
A_inverse = np.linalg.inv(A)

print("Inverse Matrix A^(-1):")
print(A_inverse)
print("-" * 30)

# --- Verification ---
# A * A_inverse should equal the Identity Matrix
identity_check = np.dot(A, A_inverse)
print("Verification (A * A_inverse = Identity):")
print(identity_check)

```

```

Original Matrix A:
[[2 0]
 [0 1]]
-----
Inverse Matrix A^(-1):
[[0.5 0. ]
 [0.  1. ]]
-----
Verification (A * A_inverse = Identity):
[[1. 0.]
 [0. 1.]]

```

```

import pandas as pd
import numpy as np

# --- Step 1: Define the Data Loader ---
# The original sklearn.datasets.load_boston is deprecated/removed.
# We use the function provided in your image to load it from the source.
def load_boston():
    data_url = "http://lib.stat.cmu.edu/datasets/boston"
    # Read the raw text data
    raw_df = pd.read_csv(data_url, sep=r"\s+", skiprows=22, header=None)
    # The data is split across lines, so we stack it to fix the shape
    data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
    target = raw_df.values[1::2, 2]
    return {'data': data, 'target': target}

# --- Step 2: Prepare the Matrices ---
boston = load_boston()
X_full = boston['data']
y_full = boston['target']

# Select first 4 rows and specific columns:
# 0 (CRIM), 4 (NOX), 5 (RM), 6 (AGE)
A = X_full[:4, [0, 4, 5, 6]]
b = y_full[:4]

feature_names = ["CRIM (Crime Rate)", "NOX (Air Pollution)", "RM (Rooms)", "

```

```

print("Matrix A (Features for 4 towns):\n", A)
print("\nVector b (Prices for 4 towns):\n", b)
print("-" * 50)

# --- Step 3: Solve for x (The Weights) ---
# We solve the linear system Ax = b
x = np.linalg.solve(A, b)

print("Calculated Weight Vector x:")
print(x)
print("-" * 50)

# --- Step 4: Interpretation ---
print("Interpretation of Results:")

# Loop through weights to check sign and interpret
for i, weight in enumerate(x):
    feature = feature_names[i]
    if weight > 0:
        relation = "direct proportion"
        direction = "increases"
    else:
        relation = "inverse proportion"
        direction = "decreases"

    print(f"* {feature}: Weight is {weight:.4f}")
    print(f"  -> The house price is in {relation} to {feature.split('(')[0]}.

```

```

Matrix A (Features for 4 towns):
[[6.320e-03 5.380e-01 6.575e+00 6.520e+01]
 [2.731e-02 4.690e-01 6.421e+00 7.890e+01]
 [2.729e-02 4.690e-01 7.185e+00 6.110e+01]
 [3.237e-02 4.580e-01 6.998e+00 4.580e+01]]

Vector b (Prices for 4 towns):
[24.  21.6 34.7 33.4]

-----
Calculated Weight Vector x:
[-3.12710043e+02 -1.15193942e+02  1.44996465e+01 -1.13259317e-01]
-----
Interpretation of Results:
* CRIM (Crime Rate): Weight is -312.7100
  -> The house price is in inverse proportion to CRIM.
* NOX (Air Pollution): Weight is -115.1939
  -> The house price is in inverse proportion to NOX.
* RM (Rooms): Weight is 14.4996
  -> The house price is in direct proportion to RM.
* AGE (Age): Weight is -0.1133
  -> The house price is in inverse proportion to AGE.

```

```

import pandas as pd
import numpy as np

# --- Step 1: Load Data (Same as before) ---
def load_boston():

```

```

data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep=r"\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]
return {'data': data, 'target': target}

boston = load_boston()
X = boston['data']
y = boston['target']

# Feature names provided in the image
feature_names = [
    "CRIM", "ZN", "INDUS", "CHAS", "NOX", "RM", "AGE", "DIS", "RAD", "TAX",
    "PTRATIO", "B", "LSTAT"
]

print(f>Data Shape: {X.shape} (506 houses, 13 features)")
print("-" * 50)

# --- Step 2: Solve using Least Squares ---
# We use np.linalg.lstsq which automatically finds the best fit w
# that minimizes the error |y - Xw|^2
w, residuals, rank, s = np.linalg.lstsq(X, y, rcond=None)

print("Calculated Weight Vector w (Best Fit):")
print(w)
print("-" * 50)

# --- Step 3: Interpretation & Comparison ---
print("1) Interpretation of Results (Full Dataset):")

for i, weight in enumerate(w):
    feature = feature_names[i]
    if weight > 0:
        relation = "direct proportion (+)"
    else:
        relation = "inverse proportion (-)"

    print(f"* {feature}: {weight: .4f} -> {relation}")

print("-" * 50)
print("2) Comparison with Question 5:")
print("In Q5, we used only 4 houses, so the math forced the line to hit thos")
print("even if it meant calculating unrealistic weights (like 'Age' being ne")
print("In Q6, we use ALL 506 houses. These weights represent the REAL genera")
print("For example, 'RM' (Rooms) typically has a strong positive weight here")

```

Data Shape: (506, 13) (506 houses, 13 features)

Calculated Weight Vector w (Best Fit):

```

[-9.28965170e-02  4.87149552e-02 -4.05997958e-03  2.85399882e+00
 -2.86843637e+00  5.92814778e+00 -7.26933458e-03 -9.68514157e-01
 1.71151128e-01 -9.39621540e-03 -3.92190926e-01  1.49056102e-02
 -4.16304471e-01]

```

1) Interpretation of Results (Full Dataset):

- * CRIM: -0.0929 -> inverse proportion (-)
- * ZN: 0.0487 -> direct proportion (+)
- * INDUS: -0.0041 -> inverse proportion (-)
- * CHAS: 2.8540 -> direct proportion (+)
- * NOX: -2.8684 -> inverse proportion (-)
- * RM: 5.9281 -> direct proportion (+)
- * AGE: -0.0073 -> inverse proportion (-)
- * DIS: -0.9685 -> inverse proportion (-)
- * RAD: 0.1712 -> direct proportion (+)
- * TAX: -0.0094 -> inverse proportion (-)
- * PTRATIO: -0.3922 -> inverse proportion (-)
- * B: 0.0149 -> direct proportion (+)
- * LSTAT: -0.4163 -> inverse proportion (-)

2) Comparison with Question 5:

In Q5, we used only 4 houses, so the math forced the line to hit those 4 points even if it meant calculating unrealistic weights (like 'Age' being negative). In Q6, we use ALL 506 houses. These weights represent the REAL general trend. For example, 'RM' (Rooms) typically has a strong positive weight here because

```
import numpy as np
from scipy.spatial.distance import cosine

# --- Step 1: Define User Rating Vectors ---
a = np.array([4, 5, 2, 2])
b = np.array([4, 0, 2, 0])
c = np.array([2, 2, 0, 1])

print("Vectors defined:")
print(f"a: {a}")
print(f"b: {b}")
print(f"c: {c}")
print("-" * 50)

# --- Step 2: Euclidean Distance ---
# Smaller value = Closer (More similar)
dist_ab = np.linalg.norm(a - b)
dist_ac = np.linalg.norm(a - c)
dist_bc = np.linalg.norm(b - c)

print("1) Euclidean Distances:")
print(f"  a <-> b: {dist_ab:.4f}")
print(f"  a <-> c: {dist_ac:.4f}")
print(f"  b <-> c: {dist_bc:.4f}")

# Find Closest and Farthest
euclid_map = {"a-b": dist_ab, "a-c": dist_ac, "b-c": dist_bc}
print(f"  >> Closest Users (Euclidean): {min(euclid_map, key=euclid_map.get)}")
print(f"  >> Farthest Users (Euclidean): {max(euclid_map, key=euclid_map.get)}")
print("-" * 50)

# --- Step 3: Cosine Distance ---
# SciPy's cosine function returns the DISTANCE (1 - Similarity)
# Smaller value = Closer (More similar taste)
```

```

cos_ab = cosine(a, b)
cos_ac = cosine(a, c)
cos_bc = cosine(b, c)

print("2) Cosine Distances:")
print(f"    a <-> b: {cos_ab:.4f}")
print(f"    a <-> c: {cos_ac:.4f}")
print(f"    b <-> c: {cos_bc:.4f}")

# Find Closest and Farthest
cosine_map = {"a-b": cos_ab, "a-c": cos_ac, "b-c": cos_bc}
print(f"    >> Closest Users (Cosine): {min(cosine_map, key=cosine_map.get)}")
print(f"    >> Farthest Users (Cosine): {max(cosine_map, key=cosine_map.get)}")

```

Vectors defined:

```

a: [4 5 2 2]
b: [4 0 2 0]
c: [2 2 0 1]

```

```

-----
1) Euclidean Distances:
    a <-> b: 5.3852
    a <-> c: 4.2426
    b <-> c: 3.6056
    >> Closest Users (Euclidean): b-c (Smallest Distance)
    >> Farthest Users (Euclidean): a-b
-----
2) Cosine Distances:
    a <-> b: 0.3611
    a <-> c: 0.0476
    b <-> c: 0.4037
    >> Closest Users (Cosine): a-c (Smallest Angle)
    >> Farthest Users (Cosine): b-c

```

```

import numpy as np

# --- Step 1: Define the Matrix ---
D = np.array([[2, 1],
              [1, 2]])

print("Matrix D:")
print(D)
print("-" * 30)

# --- Step 2: Calculate Eigenvalues ---
# np.linalg.eig returns two things: eigenvalues and eigenvectors.
# We only need the eigenvalues for this question.
eigenvalues, eigenvectors = np.linalg.eig(D)

print("Calculated Eigenvalues:")
print(eigenvalues)

# --- Verification ---
print("-" * 30)
print("Manual Check:")
print("Roots of  $(\lambda - 3)(\lambda - 1) = 0$  are indeed 3 and 1.")

```

Matrix D:

```
[[2 1]
 [1 2]]
```

Calculated Eigenvalues:

```
[3. 1.]
```

Manual Check:

Roots of $(\lambda - 3)(\lambda - 1) = 0$ are indeed 3 and 1.

```
import math
```

```
# --- Step 1: Define Given Values ---
```

```
log_2_given = 0.69
```

```
log_3_given = 1.10
```

```
print(f"Given: log(2) = {log_2_given}")
```

```
print(f"Given: log(3) = {log_3_given}")
```

```
print("-" * 30)
```

```
# --- Step 2: Calculate log(12) manually ---
```

```
# Formula: log(12) = 2 * log(2) + log(3)
```

```
log_12_manual = (2 * log_2_given) + log_3_given
```

```
print(f"Calculated log(12) using formula: {log_12_manual:.2f}")
```

```
print("-" * 30)
```

```
# --- Step 3: Verification with Python's math library ---
```

```
# Note: The values 0.69 and 1.10 correspond to the Natural Log (ln)
```

```
real_log_12 = math.log(12)
```

```
print(f"Actual natural log(12) via Python: {real_log_12:.4f}")
```

```
print("The calculated approximation (2.48) is very close to the actual value
```

```
Given: log(2) = 0.69
```

```
Given: log(3) = 1.1
```

```
-----  
Calculated log(12) using formula: 2.48
```

```
-----  
Actual natural log(12) via Python: 2.4849
```

```
The calculated approximation (2.48) is very close to the actual value (2.4849
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# --- Step 1: Define Functions ---
```

```
def logistic(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
def inverse_logistic(y):
```

```
    # This is the formula we derived: ln(y / (1 - y))
```

```
    return np.log(y / (1 - y))
```

```
# --- Step 2: Verification with a specific value ---
```

```

x_original = 2.5
y_val = logistic(x_original)
x_recovered = inverse_logistic(y_val)

print(f"1) Original x: {x_original}")
print(f"2) Logistic(x) -> y: {y_val:.4f}")
print(f"3) Inverse(y) -> x: {x_recovered:.4f}")
print(f"    Matches original? {np.isclose(x_original, x_recovered)}")
print("-" * 50)

# --- Step 3: Visualization (Optional but helpful) ---
# We plot the Logistic function and its Inverse to show symmetry
x_vals = np.linspace(-6, 6, 100)
y_vals = logistic(x_vals)

# For inverse, input range is (0, 1) exclusive
y_inputs = np.linspace(0.01, 0.99, 100)
x_outputs = inverse_logistic(y_inputs)

plt.figure(figsize=(10, 5))

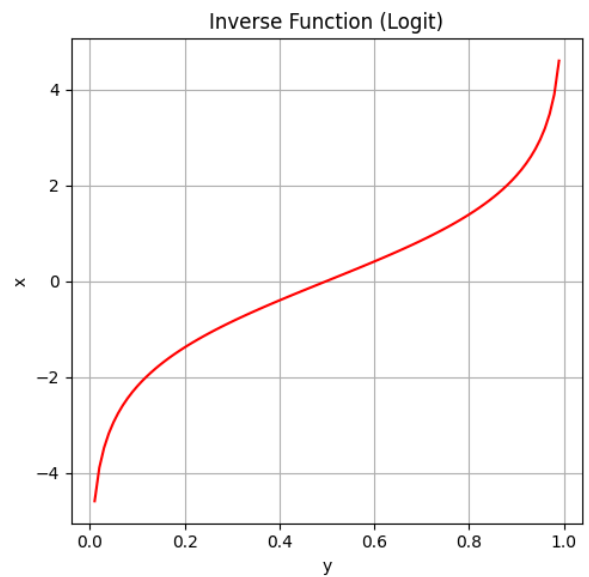
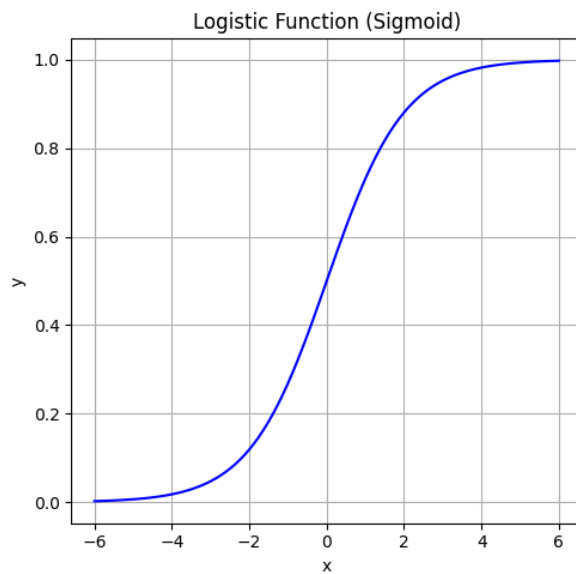
# Plot Logistic
plt.subplot(1, 2, 1)
plt.plot(x_vals, y_vals, 'b')
plt.title("Logistic Function (Sigmoid)")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)

# Plot Inverse (Logit)
plt.subplot(1, 2, 2)
plt.plot(y_inputs, x_outputs, 'r')
plt.title("Inverse Function (Logit)")
plt.xlabel("y")
plt.ylabel("x")
plt.grid(True)

plt.tight_layout()
plt.show()

```

1) Original x: 2.5
2) Logistic(x) -> y: 0.9241
3) Inverse(y) -> x: 2.5000
Matches original? True



```
import sympy as sp

# --- Step 1: Define Symbols ---
# We define x as the variable, and k, a, b as constants (real numbers)
x, k, a, b = sp.symbols('x k a b', real=True)

# --- Step 2: Define the Functions ---
f_A = x**3 - 1
f_B = sp.log(x**2 - 3*k)
f_C = sp.exp(a * x**b)

print("Functions Defined:")
print(f"A {f_A}")
print(f"B {f_B}")
print(f"C {f_C}")
print("-" * 50)

# --- Step 3: Differentiate ---
# sp.diff(function, variable) calculates the derivative
diff_A = sp.diff(f_A, x)
diff_B = sp.diff(f_B, x)
diff_C = sp.diff(f_C, x)
```

```

print("Calculated Derivatives:")
print(f"A') {diff_A}")
print(f"B') {diff_B}")
print(f"C') {diff_C}")

```

Functions Defined:

```

A) x**3 - 1
B) log(-3*k + x**2)
C) exp(a*x**b)

```

Calculated Derivatives:

```

A') 3*x**2
B') 2*x/(-3*k + x**2)
C') a*b*x**b*exp(a*x**b)/x

```

```

import sympy as sp

```

--- Step 1: Define Symbols and Function ---

```

x, y = sp.symbols('x y', real=True)
f = sp.exp(x**2 + 2*y**2)

```

```

print(f"Function f(x,y): {f}")
print("-" * 50)

```

--- Step 2: First Order Partial Derivatives ---

```

# diff(function, variable_to_differentiate_by)
fx = sp.diff(f, x)
fy = sp.diff(f, y)

```

print("1) First Derivatives:")

```

print(f"    fx: {fx}")
print(f"    fy: {fy}")
print("-" * 50)

```

--- Step 3: Second Order Partial Derivatives ---

```

# diff(function, var1, var2) -> differentiates by var1 then var2
fxx = sp.diff(f, x, x)
fxy = sp.diff(f, x, y)
fyx = sp.diff(f, y, x)
fyy = sp.diff(f, y, y)

```

print("2) Second Derivatives:")

```

# We use sp.simplify() to make the output look cleaner (factoring terms)
print(f"    fxx: {sp.simplify(fxx)}")
print(f"    fxy: {sp.simplify(fxy)}")
print(f"    fyx: {sp.simplify(fyx)}")
print(f"    fyy: {sp.simplify(fyy)}")

```

Function f(x,y): exp(x**2 + 2*y**2)

1) First Derivatives:

```

    fx: 2*x*exp(x**2 + 2*y**2)
    fy: 4*y*exp(x**2 + 2*y**2)

```

2) Second Derivatives:

```
fxx: (4*x**2 + 2)*exp(x**2 + 2*y**2)
fxy: 8*x*y*exp(x**2 + 2*y**2)
fyx: 8*x*y*exp(x**2 + 2*y**2)
fyy: (16*y**2 + 4)*exp(x**2 + 2*y**2)
```

```
import sympy as sp

# --- Step 1: Define the Symbols ---
# 'x' is our variable.
# 'k', 'a', 'b' are constants.
x = sp.symbols('x')
k, a, b = sp.symbols('k a b')

# --- Step 2: Define the Functions ---
# A:  $f(x) = x^3 - 1$ 
f_A = x**3 - 1

# B:  $f(x) = \log(x^2 - 3k)$ 
# Note: In SymPy, 'log' is the natural logarithm (ln).
f_B = sp.log(x**2 - 3*k)

# C:  $f(x) = \exp(ax^b)$ 
f_C = sp.exp(a * (x**b))

print("defined functions:")
print(f"A) {f_A}")
print(f"B) {f_B}")
print(f"C) {f_C}")
print("-" * 50)

# --- Step 3: Differentiate using SymPy ---
# Syntax: sp.diff(function, variable)

derivative_A = sp.diff(f_A, x)
derivative_B = sp.diff(f_B, x)
derivative_C = sp.diff(f_C, x)

print("Calculated Derivatives:")
print(f"A') {derivative_A}")
print(f"B') {derivative_B}")
print(f"C') {derivative_C}")

defined functions:
A)  $x^3 - 1$ 
B)  $\log(-3k + x^2)$ 
C)  $\exp(ax^b)$ 
-----
Calculated Derivatives:
A')  $3x^2$ 
B')  $2x/(-3k + x^2)$ 
C')  $a*b*x^{b-1}*\exp(ax^b)/x$ 
```